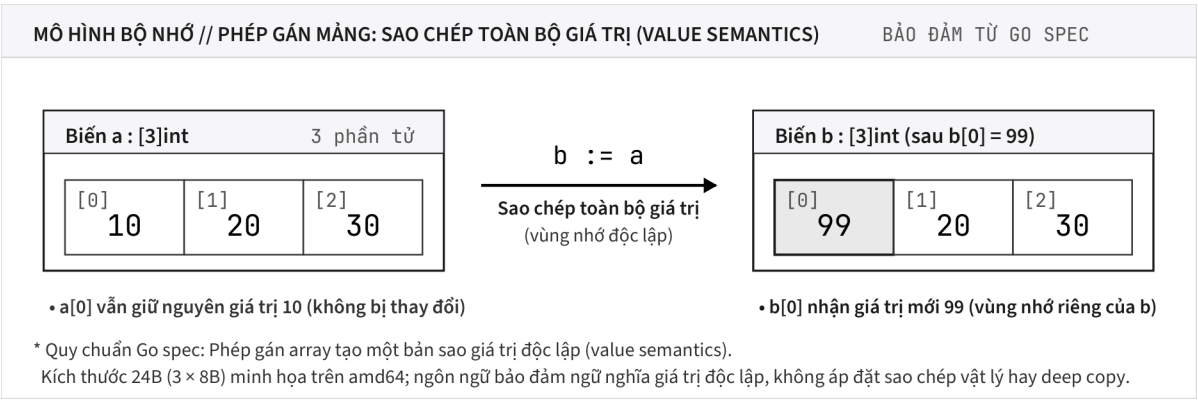
CHƯƠNG 02: BẢN DUYỆT SƠ ĐỒ KỸ THUẬT (TECHNICAL DIAGRAM SPECIMEN)

Hiển thị 100% kích thước xuất bản (448.22 pt) • Phong cách đen trắng tối giản • Chuẩn Go spec vs gc runtime

HÌNH 2.1 — MÔ HÌNH BỘ NHỚ // PHÉP GÁN MẢNG: SAO CHÉP TOÀN BỘ GIÁ TRỊ

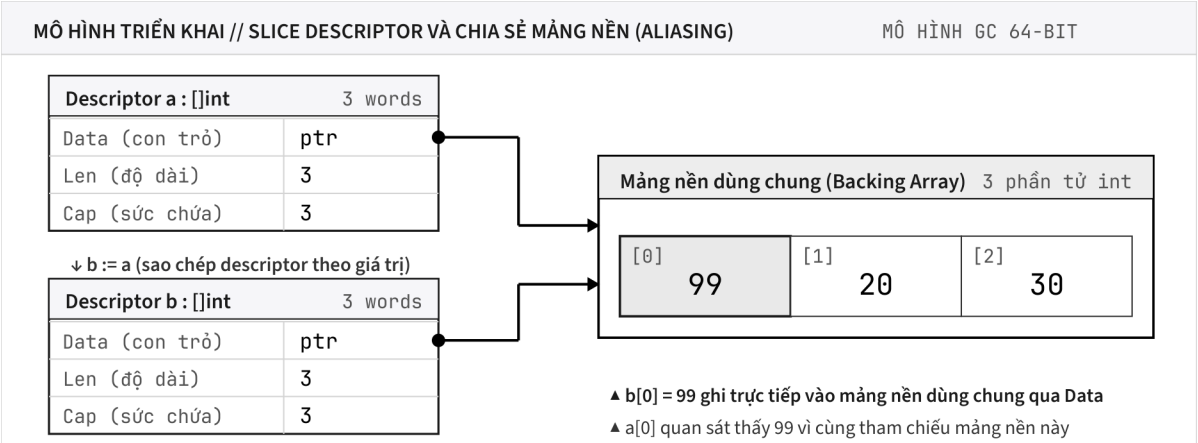
Hai array value sau assignment: phần tử được copy vào vùng dữ liệu riêng (bảo đảm ngữ nghĩa giá trị độc lập từ Go spec).



* Quy chuẩn Go spec: Phép gán array tạo một bản sao giá trị độc lập (value semantics).
Kích thước 24B (3 × 8B) minh họa trên amd64; ngôn ngữ bảo đảm ngữ nghĩa giá trị độc lập, không áp đặt sao chép vật lý hay deep copy.

HÌNH 2.2 — MÔ HÌNH TRIỂN KHAI // SLICE DESCRIPTOR VÀ CHIA SẺ MẢNG NỀN (ALIASING)

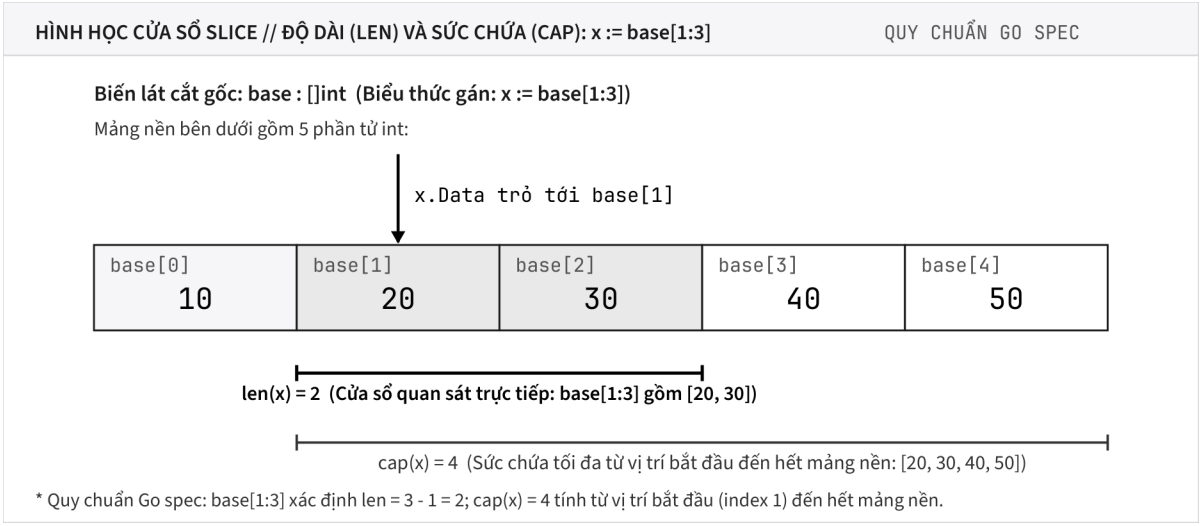
Hai slice value sau assignment: descriptor được copy theo giá trị, backing storage được chia sẻ (mô hình gc 64-bit).



* Quy chuẩn Go spec: Slice là cửa sổ tham chiếu mảng nền; cấu trúc Data/Len/Cap là mô hình triển khai của gc 64-bit.
Phép gán b := a sao chép descriptor theo giá trị; cả hai descriptor cùng tham chiếu chung một backing array.

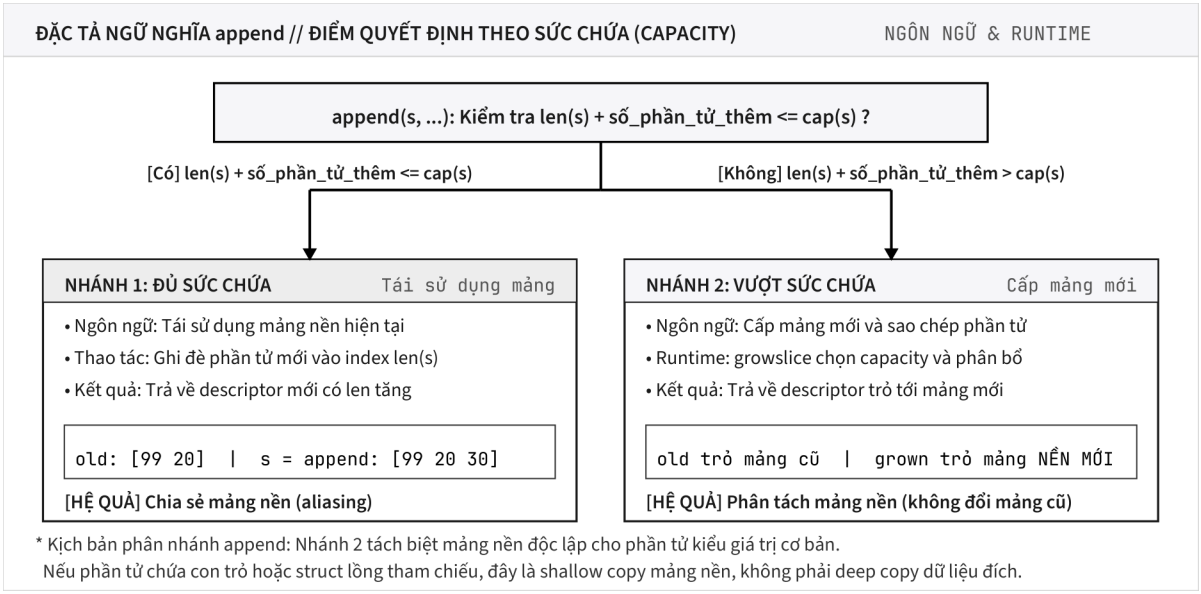
HÌNH 2.3 — HÌNH HỌC CỬA SỔ SLICE // ĐỘ DÀI (LEN) VÀ SỨC CHỨA (CAP)

Một slice window trên mảng nền của lát cắt gốc base : []int; vùng thấy được là len(x) = 2; sức chứa tối đa là cap(x) = 4.



HÌNH 2.4 — ĐẶC TẢ PHÂN NHÁNH append // ĐIỂM QUYẾT ĐỊNH THEO SỨC CHỨA (CAPACITY)

Hai kết quả của append: còn capacity (len(s) + số_phần_tử_thêm <= cap(s)) thì tái sử dụng mảng nền; vượt capacity thì cấp mảng mới (growslice).



* Kịch bản phân nhánh append: Nhánh 2 tách biệt mảng nền độc lập cho phần tử kiểu giá trị cơ bản.
Nếu phần tử chứa con trỏ hoặc struct lồng tham chiếu, đây là shallow copy mảng nền, không phải deep copy dữ liệu đích.